MicroPascal-parte1

Universidade Católica de Pernambuco
Disciplina: Construção de Compiladores
Professor: Robson Lins

Projeto de Compiladores – Segunda Parte

O objetivo desta parte do projeto é estender a primeira parte do projeto acrescentando duas das três etapas seguintes: *análise semântica* e *geração de código intermediário*. **Esta parte projeto tem peso 5, ou seja, valerá metade da nota do 2º GQ**. As equipes devem, obrigatoriamente, ter 5 ou 6 integrantes.

A seção 1, descreve uma adaptação necessária ao projeto, se ainda não foi implementada. Nas seções 2 e 3, estão detalhados os requisitos de cada etapa da compilação que deve ser implementada no projeto.

1. Geração da Árvore Sintática

Vocês devem lembrar que a análise sintática identifica a árvore sintática. Porém isso acontece em um sentido abstrato, conceitual. O que esta parte do seu compilador deve fazer é criar classes (ou structs) que representem essa árvore de modo mais concreto.

Durante a análise sintática, essas classes deverão ser instanciadas de modo que cada objeto represente um trecho importante do código fonte em Micro-Pascal dado como entrada. Com a árvore construída assim, fica mais fácil desenvolver as próximas etapas da compilação.

O uso da árvore favorece a modularização, deixando o seu projeto com melhor qualidade de código. Veja nos slides das aulas as dicas para representar a árvore adequadamente.

2. Análise Semântica

O analisador semântico faz verificações adicionais, que não podem ser feitas na análise sintática. Além disso, ela reune mais informações sobre o código, que podem ser usadas nas próximas etapas.

Abaixo, dividimos os requisitos para esta etapa em: verificações de escopo, verificações de tipos em expressões e verificações de tipos em comandos.

2.1 Verificação de Escopo

O projeto deve seguir regras de escopo parecidas com as de Pascal:

1) Não pode haver duas declarações de variáveis com o mesmo nome. Se houver, o compilador deve indicar um erro.

2) Toda variável precisa ser declarada para poder ser usada. Se uma variável for usada sem ser declarada, o compilador deve indicar um erro.

O tratamento dos nomes e dos seus escopos deve ser feito por meio de uma tabela de símbolos. Cada entrada da tabela deve conter, pelo menos, uma associação entre o nome e o seu tipo. Informações adicionais podem ser guardadas na tabela, se necessário.

2.2 Verificações de Tipos em Expressões

O compilador deve analisar cada expressão e, com base nas suas sub-expressões, decidir o tipo dela, se as operações entre os tipos forem válidas. Porém, se uma operação for aplicada a operandos de tipos inválidos, o compilador deve indicar um erro de tipo.

Como você deve saber, a linguagem suporta os tipos numéricos **inteiro** (*integer*) e **real** (*real*), além do tipo **caractere** (*character*). Adicionalmente, o compilador deve representar um tipo especial **booleano** que permite os valores verdadeiro e falso. Esse tipo é usado apenas internamente pelo compilador (programadores de Micro-Pascal não podem declarar variáveis desse tipo).

As regras de tipo de Micro-Pascal são:

1) As operações de **adição**, **subtração** e **multiplicação** só se aplicam a operandos numéricos de mesmo tipo. Se ambos os operandos forem inteiros, a expressão deve ser considerada do tipo inteiro. Se ambos forem reais, o a expressão é considerada do tipo real.

2) Também é permitida a **adição** e a **subtração** entre um caracter e um inteiro (nesta ordem), dando um caracter como resultado. Para qualquer outra combinação de operandos, o compilador deve indicar erro.

3) A **divisão de inteiros "div"** só aceita operandos do tipo inteiro e seu resultado é também um inteiro. Já a **divisão de reais "/"**, como o nome indica, só aceita reais como operandos e dá um resultado do tipo real.

4) As operações de **comparação** <, <=, >, >= só se aplicam a operandos numérico de mesmo tipo (inteiro com inteiro ou real com real), dando como resultado um valor do tipo booleano. Para qualquer outra combinação de tipos, o compilador deve indicar erro.

5) As operações de **comparação** = e <> só se aplicam a operandos do mesmo tipo, inclusive caracteres e booleanos. Para operandos com tipos mistos, o compilador deve indicar erro.

6) As **operações lógicas "and"** e **"or"** só se aplicam a operandos booleanos, dando um booleano como resultado.

7) O **menos unário** só pode ser aplicado a um operando numérico. Se o operando for inteiro, a expressão toda tem tipo inteiro. Se o operando for do tipo real, a expressão é do tipo real. Se o operando for de qualquer outro tipo, o compilador deve retornar erro.

2.3 Verificações de Tipos em Comandos

Para os comandos que usam expressões, o compilador apenas irá verificar se o tipo delas é adequado. Se não for, o compilador deve indicar erro.

1) Os comandos de atribuição devem possuir, no lado esquerdo, uma variável de mesmo tipo que a expressão no lado direito (cujo tipo já deve ter sido verificado).

2) Já as expressões presentes nos comandos de decisão e de iteração devem ser do tipo booleano.

3. Geração de Código Intermediário

Esta etapa gera um código formado por instruções simples, parecidas com códigos em linguagens assembly ou com instruções de máquina (que seriam o código final).

Nesta etapa, é fundamental entender o significado dos comandos (atribuições, if-then-else, while, repeat-unitl), que vocês já devem entender bem devido ao conhecimento de programação que possuem. Qualquer dúvida no entendimento da semântica (significado) da linguagem, entrem em contato comigo.

Abaixo, estão listados os principais requisitos:

1) O projeto deverá gerar um **código de três endereços** (visto em aula) equivalente ao código Micro-Pascal dado como entrada.

2) Quando for preciso gerar variáveis temporárias, usar **nomes distintos** das variáveis declaradas em código. Para garantir isso, pode ser necessário consultar a tabela de símbolos.

3) Ao gerar código para avaliar as expressões: é obrigatório **obedecer as precedências** definidas na primeira parte do projeto. Para isso, basta seu *parser* estar tratando essas precedências corretamente. Se o seu *parser* não estiver tratando isso adequadamente, corrija-o.